

history is a list of lists. The outer list is the 100 agents in the simulation, and each inner list is the state of that agent at each of the 200 time points in the sim. My counts are returning empty unexpectedly

Vander_Wood_BMI500_HW11.ipynb • Cell 0:8-15

> Used 1 reference

> Explaining count aggregation issue and fix

Your accumulator lists never get any elements, so `susceptible_counts[i] += 1` hits an empty list. Size them up front (based on one agent's history) before you loop:

```
# ...existing code...
timesteps = len(history[0])
susceptible_counts = [0] * timesteps
infected_counts = [0] * timesteps
recovered_counts = [0] * timesteps

for agent_history in history:
    for i, state in enumerate(agent_history):
        if state == 'S':
            susceptible_counts[i] += 1
        elif state == 'I':
            infected_counts[i] += 1
        elif state == 'R':
            recovered_counts[i] += 1
# ...existing code...
```

Now I'm seeing what looks like one agent's history plotted, rather than the summation of all of them

Vander_Wood_BMI500_HW11.ipynb • Cell 0:1-24

Used 1 reference

Correcting simulation history structure

`runSimulation` currently appends one big list containing every agent's history, so the plotting loop sees that first wrapper and ends up graphing a single agent. Return the per-agent histories directly:

```
# ...existing code...
def runSimulation(agents, steps, grid):
    for _ in range(steps):
        for i in range(len(grid)):
            for j in range(len(grid[i])):
                grid[i][j] = None

        for agent in agents:
            agent.move()
            x, y = agent.get_position()
            grid[x][y] = agent

        for i, row in enumerate(grid):
            for j, cell in enumerate(row):
                if cell is None:
                    continue
                local_agents = [a for a in agents if a.get_position() == (i, j)]
                infected_present = any(a.get_status() == 'I' for a in local_agents)
                for agent in local_agents:
                    agent.update_status(exposed_yn=infected_present)
    return [agent.get_history() for agent in agents]
# ...existing code...
```



I'm currently using constants.py to store global variables. However, I'd like to be able to run a variety of simulations with different variables (adjusting P and Q or the starting number of infected, for example). How could I restructure to be able to more easily do this?

constants.py

simulation.py

Vander_Wood_BMI500_HW11.ipynb • Cell 0:1-3

> Used 3 references

> Suggesting configuration object restructure

Replace the global module with a configuration object that you pass into every function. For example, define a `SimulationConfig` dataclass (fields: `grid_size`, `steps`, `agent_count`, `p`, `q`, `social_distancing`). Instantiate it for each scenario, then call `makeGrid(config.grid_size)`, `makeAgents(config.agent_count, config.initial_infected_fraction)`, and `runSimulation(agents, config.steps, grid, config)`. This keeps parameters together and lets you run batches by creating multiple config instances.